

**SCHOOL OF
COMPUTING**

DESIGN AND ANALYSIS OF ALGORITHMS

LAB WORKBOOK

WEEK - 7

NAME : Kurra Venkata Gokul

ROLL NO : CH.SC.U4CSE24121

CLASS : CSE-B

Question 1: Let there be 14 jobs with the profit of
22,19,29,28,30,21,27,25,24,26,14,27,19,11 with deadlines
3,3,8,6,7,5,10,4,6,12,13,2,14,1

Implement the greedy algorithm for the Job Sequencing with Deadlines and
determine the optimal sequence of jobs that maximizes total profit.

Job Sequencing (Greedy method)

Q. Let there be 14 jobs with the profit of 22, 19, 29, 28, 30, 21, 27, 25, 24, 26, 14, 27, 19, 11 with deadlines - 3, 3, 8, 6, 7, 5, 10, 4, 6, 12, 13, 2, 14, 1

Given: Number of Jobs (n) = 14

Profits corresponding to jobs J_1 to J_{14} are P_1 to P_{14}

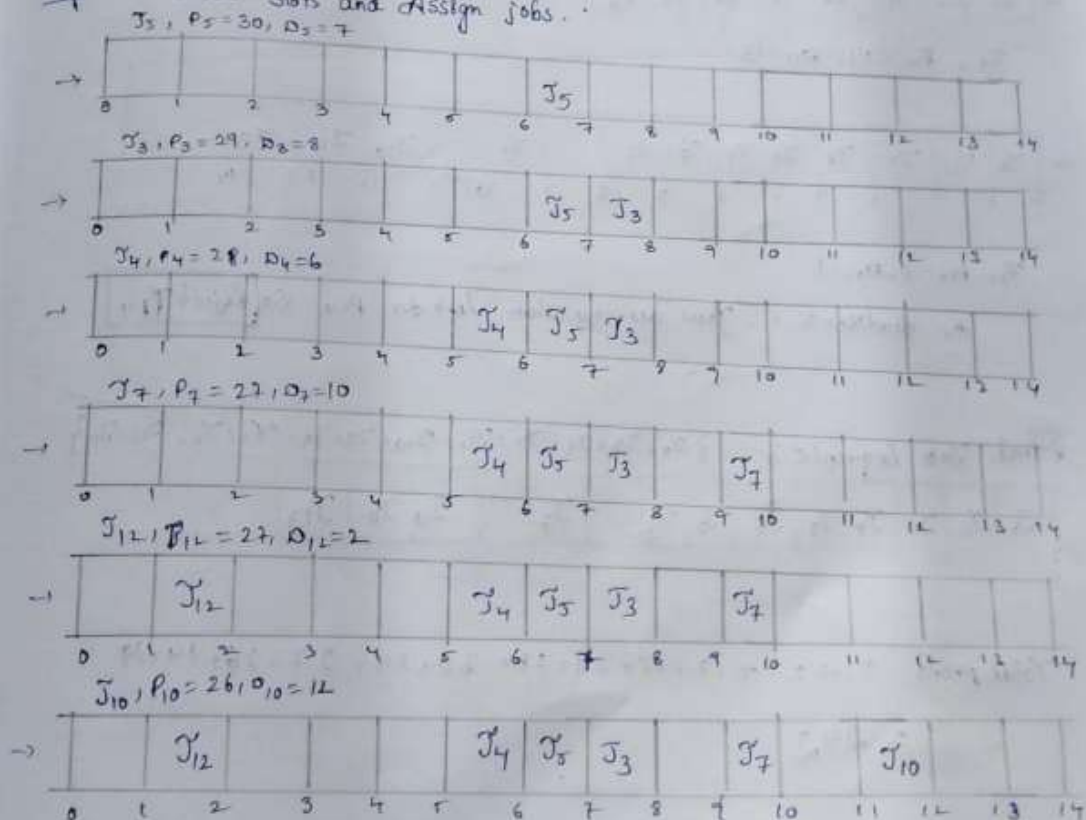
$(P_1 \text{ to } P_{14}) = (22, 19, 29, 28, 30, 21, 27, 25, 24, 26, 14, 27, 19, 11)$

$(D_1 \text{ to } D_{14}) = (3, 3, 8, 6, 7, 5, 10, 4, 6, 12, 13, 2, 14, 1)$

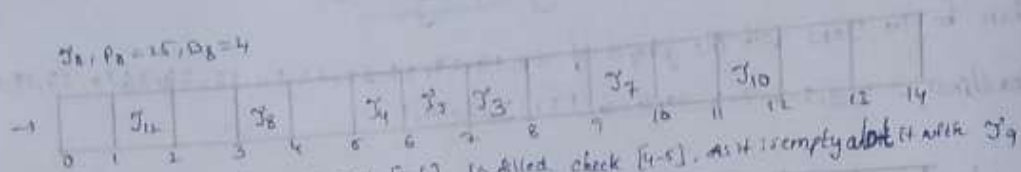
Step 1: Arrange the jobs in descending order based on profits and write corresponding deadlines.

30	29	28	27	27	26	25	24	22	21	19	19	14	11
7	8	6	10	2	12	4	6	3	5	8	14	12	1
J_5	J_3	J_4	J_7	J_{12}	J_{10}	J_8	J_9	J_1	J_6	J_2	J_{13}	J_{11}	J_{14}

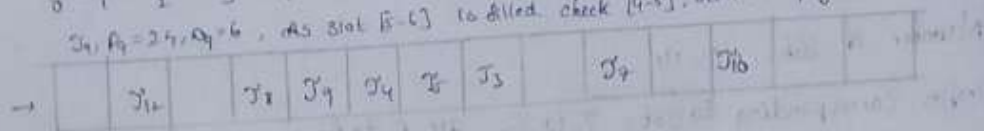
Step 2: Create slots and Assign jobs.



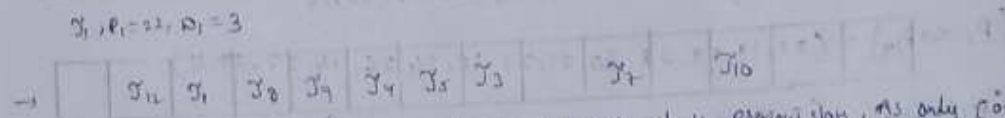
$$J_8, P_8=15, D_8=4$$



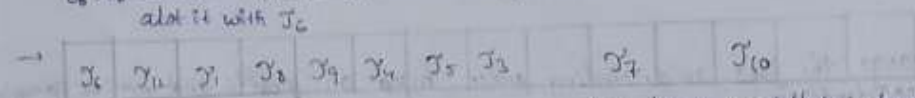
$$J_9, P_9=27, D_9=6$$



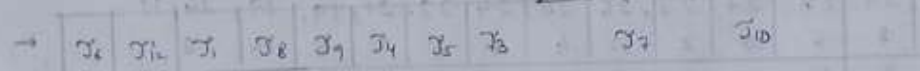
$$J_1, P_1=22, D_1=3$$



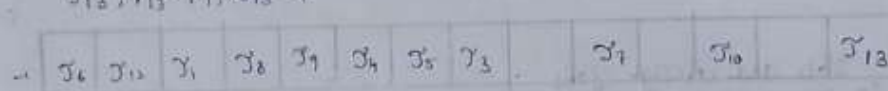
$J_6, P_6=21, D_6=5$, As slot 5-6 is already assigned, check previous slots, as only slot 3 is free slot 4 with J_6



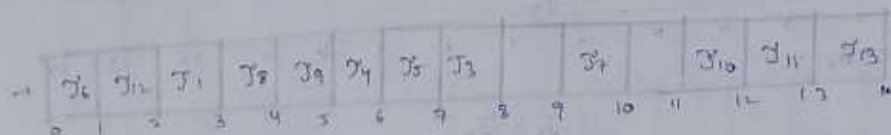
$J_2, P_2=19, D_2=3$. All slots before deadline i.e. 3 are filled already. So no slot for J_2 . Reject J_2



$$J_{13}, P_{13}=19, D_{13}=14$$



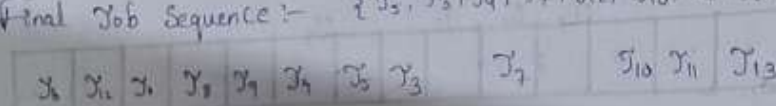
$$J_4, P_{11}=14, D_{11}=13$$



$$J_{14}, P_{14}=11, D_{14}=1$$

As deadline is 1, there are no slots left for P_{14} . So Reject J_{14}

Final Job Sequence :- $\{J_3, J_3, J_4, J_7, J_{12}, J_{10}, J_8, J_9, J_1, J_6, J_{13}, J_{11}\}$



$$\text{Total profit} = 21 + 27 + 22 + 25 + 24 + 22 + 30 + 29 + 27 + 26 + 14 + 19$$

$$= 292$$

CODE:

```
//CH.SC.U4CSE24121
/* Job Sequencing using Greedy Method */

#include <stdio.h>
#define LIMIT 100

struct Task
{
    int taskId;
    int gain;
    int dueTime;
};

void sortTasks(struct Task taskList[], int count)
{
    int x, y;
    struct Task swapTemp;

    for(x = 0; x < count - 1; x++)
    {
        for(y = 0; y < count - x - 1; y++)
        {
            if(taskList[y].gain < taskList[y + 1].gain)
            {
                swapTemp = taskList[y];
                taskList[y] = taskList[y + 1];
                taskList[y + 1] = swapTemp;
            }
        }
    }
}
```

```
int getMaxDeadline(struct Task taskList[], int count)
{
    int index, maxDeadline = taskList[0].dueTime;

    for(index = 1; index < count; index++)
    {
        if(taskList[index].dueTime > maxDeadline)
        {
            maxDeadline = taskList[index].dueTime;
        }
    }
    return maxDeadline;
}

int main()
{
    struct Task taskList[LIMIT];
    int count, i, k;

    printf("Enter number of tasks: ");
    scanf("%d", &count);

    printf("Enter profits:\n");
    for(i = 0; i < count; i++)
    {
        taskList[i].taskId = i + 1;
        scanf("%d", &taskList[i].gain);
    }

    printf("Enter deadlines:\n");
    for(i = 0; i < count; i++)
    {
        scanf("%d", &taskList[i].dueTime);
    }
}
```

```

    sortTasks(taskList, count);

    int lastSlot = getMaxDeadline(taskList, count);
    int schedule[LIMIT];

    for(i = 1; i <= lastSlot; i++)
    {
        schedule[i] = -1;
    }

    int maxProfit = 0;

    for(i = 0; i < count; i++)
    {
        for(k = taskList[i].dueTime; k >= 1; k--)
        {
            if(schedule[k] == -1)
            {
                schedule[k] = taskList[i].taskId;
                maxProfit += taskList[i].gain;
                break;
            }
        }
    }

    printf("\nSlot Arrangement:\n");
    for(i = 1; i <= lastSlot; i++)
    {
        if(schedule[i] == -1)
            printf("Slot %d : _\n", i);
        else
            printf("Slot %d : T%d\n", i, schedule[i]);
    }

    printf("\nMaximum Profit = %d\n", maxProfit);

    return 0;
}

```

OUTPUT:

```
Enter number of tasks: 14
Enter profits:
22 19 29 28 30 21 27 25 24 26 14 27 19 11
Enter deadlines:
3 3 8 6 7 5 10 4 6 12 13 2 14 1

Slot Arrangement:
Slot 1 : T6
Slot 2 : T12
Slot 3 : T1
Slot 4 : T8
Slot 5 : T9
Slot 6 : T4
Slot 7 : T5
Slot 8 : T3
Slot 9 : _
Slot 10 : T7
Slot 11 : _
Slot 12 : T10
Slot 13 : T11
Slot 14 : T13

Maximum Profit = 292
```

Time Complexity:

1. Sorting the jobs by profit

We used Bubble Sort in the program.

Time complexity: $O(n^2)$

2. Finding maximum deadline

We check all jobs once.

Time complexity: $O(n)$

3. Assigning jobs to slots

For each job, we may check up to d slots. $O(n^2)$

Total Time Complexity

$$O(n^2) + O(n) + O(n^2) = O(n^2)$$

Space Complexity

- Job array $\rightarrow O(n)$
- Slot array $\rightarrow O(d)$

Total Space: $O(n)$